

El juego de la vida de Conway: Una implementación paralela

Fajardo Ana Paloma
Falcón Diaz Ricardo
Monroy Alarcón Omar Ulises
Perez Lopez Zaira Cecilia
Ramírez Rodríguez Enrique

Universidad Nacional Autónoma de México
Facultad de Estudios Superiores Acatlán

22 de mayo de 2026

1 Introducción

2 Implementación paralela

3 Trabajo futuro

Una célula **muerta** con exactamente tres vecinas **vivas** se convierte en **viva**.

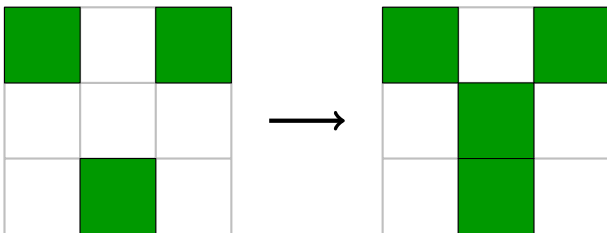


Figura 1: Nacimiento de una célula

Una célula **viva** con dos o tres vecinas vivas **vivas** permanece **viva**

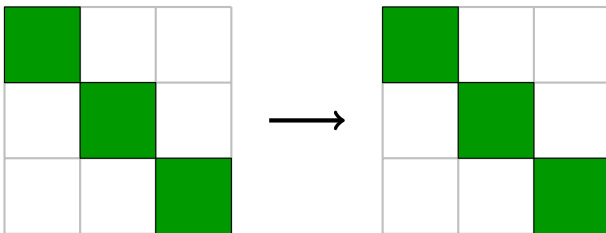


Figura 2: Supervivencia de una célula

Muerte por aislamiento

Una célula **viva** con menos de dos vecinas **vivas** muere.

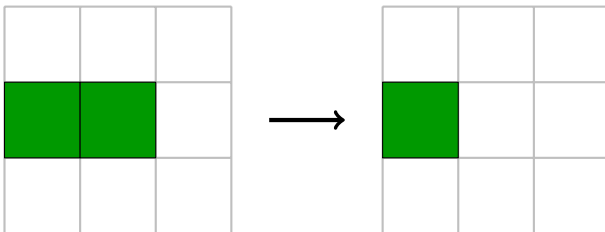


Figura 3: Muerte de una célula por aislamiento

Muerte por sobre población

Una célula **viva** con más de tres vecinas **vivas** muere.

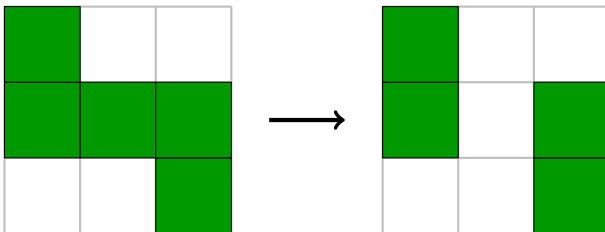
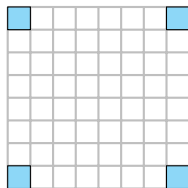


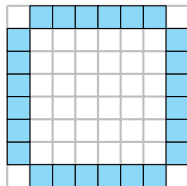
Figura 4: Muerte de una célula por sobre población

obtener_vecinos

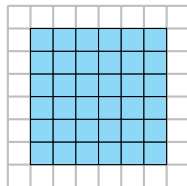
Para nuestra implementación dividiremos el tablero en 3 partes:



Esquinas



Orillas



Interior

```
1 def obtener_vecinos(tablero, f, c):
2     vivos = 0
3     f_inicio, f_fin = max(0, f - 1), min(FILAS - 1, f + 1)
4     c_inicio, c_fin = max(0, c - 1), min(COLS - 1, c + 1)
5
6     for i in range(f_inicio, f_fin + 1):
7         for j in range(c_inicio, c_fin + 1):
8             if i == f and j == c: continue
9             if tablero[i, j] == 1: vivos += 1
10
11     return vivos
```

```
1 def procesar_bloque(args):
2     f_idx_inicio, f_idx_fin, c_idx_inicio, c_idx_fin, tablero = args
3     alto, ancho = f_idx_fin - f_idx_inicio, c_idx_fin - c_idx_inicio
4     resultado_bloque = np.zeros((alto, ancho), dtype=int)
5     for f in range(f_idx_inicio, f_idx_fin):
6         for c in range(c_idx_inicio, c_idx_fin):
7             vecinos = obtener_vecinos(tablero, f, c)
8             if (tablero[f, c] == 1 and vecinos in [2, 3]) or (tablero[f, c] == 0 and
9                 vecinos == 3):
10                 resultado_bloque[f - f_idx_inicio, c - c_idx_inicio] = 1
11     return (f_idx_inicio, f_idx_fin, c_idx_inicio, c_idx_fin, resultado_bloque)
```



```
1 def crear_tareas(num_procesos, tablero):
2     if num_procesos == 1:
3         return [(0, 1000, 0, 1000, tablero)]
4     if num_procesos == 2:
5         return [(0, 1000, 0, 500, tablero), (0, 1000, 500, 1000, tablero)]
6     if num_procesos == 4:
7         return [(i, i+500, j, j+500, tablero) for i in [0, 500] for j in [0, 500]]
8     if num_procesos == 8:
9         return [(i, i+250, j, j+500, tablero) for i in range(0, 1000, 250) for j in
                [0, 500]]
```

ejecutar_prueba

```
1 def ejecutar_prueba(n_procesos):
2     np.random.seed(SEMILLA)
3     tablero = np.random.choice([0, 1], size=(FILAS, COLS), p=[0.95, 0.05])
4     tiempos = []
5
6     print(f"\n>>> INICIANDO PRUEBA CON {n_procesos} PROCESO(S) <<<")
7
8     with Pool(processes=n_procesos) as pool:
9         for g in range(MAX_GENS):
10             inicio = time.perf_counter()
11
12             tareas = crear_tareas(n_procesos, tablero)
13             resultados = pool.map(procesar_bloque, tareas)
14
15             nuevo_tablero = np.zeros_like(tablero)
16             for f_s, f_e, c_s, c_e, bloque in resultados:
17                 nuevo_tablero[f_s:f_e, c_s:c_e] = bloque
18
19             t_gen = time.perf_counter() - inicio
20             tiempos.append(t_gen)
21             tablero = nuevo_tablero
22
23             poblacion = np.sum(tablero)
24             print(f" [Gen {g+1}] Celulas vivas: {poblacion} | Tiempo: {t_gen:.4f}s")
25
26     return sum(tiempos)
```

Calculo de estados mediante *slicing*

```
1 def actualizar_tablero(tablero):
2     nuevo_tablero = np.zeros_like(tablero)
3
4     vecinos = (
5         tablero[:-2, :-2] + tablero[:-2, 1:-1] + tablero[:-2, 2:] +
6         tablero[1:-1, :-2] + tablero[1:-1, 2:] +
7         tablero[2:, :-2] + tablero[2:, 1:-1] + tablero[2:, 2:]
8     )
9
10    sub_tablero = tablero[1:-1, 1:-1]
11
12    supervive = (sub_tablero == 1) & ((vecinos == 2) | (vecinos == 3))
13    nace = (sub_tablero == 0) & (vecinos == 3)
14
15    nuevo_tablero[1:-1, 1:-1] = np.where(supervive | nace, 1, 0)
16
17    return nuevo_tablero
```

Visualización con Pygame

```
1 def dibujar(pantalla, tablero):
2     pantalla.fill(COLOR_BG)
3
4     filas_vivas, cols_vivas = np.where(tablero == 1)
5
6     for f, c in zip(filas_vivas, cols_vivas):
7         rectangulo = (c * TAMANO_CELDA, f * TAMANO_CELDA, TAMANO_CELDA - 1,
8                        TAMANO_CELDA - 1)
9         pygame.draw.rect(pantalla, COLOR_CELDA, rectangulo)
```